

# 🎙 Oral ImmoDash — Guide complet (data · pipeline · code · storytelling)

**Objectif** : pouvoir expliquer **avec tes mots** la data, le flux complet et le code cœur. Le prof pénalise « l'IA sans comprendre » → pour chaque bloc de code, retiens **CE QUE ça fait** et **POURQUOI**. En démo : si on te pose une question, **ouvre le fichier et montre la fonction**.

## 1) LE STORYTELLING DE LA DONNÉE (le fil rouge à raconter)

**L'accroche (30 s)** : > « Chaque vente immobilière en France est publiée en open data : les DVF, *Demandes de Valeurs Foncières*. Sur 3 ans, c'est **2,79 millions de ventes**. Brut, c'est illisible. ImmoDash répond > en 10 secondes à : **où achète-t-on, à quel prix, et comment ça évolue ?** »

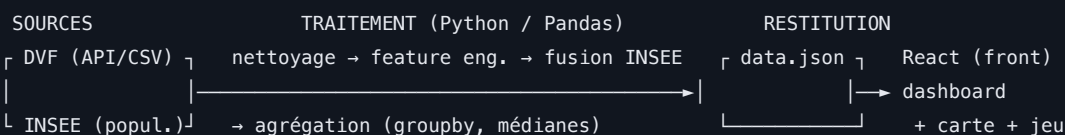
**L'histoire en 4 temps (à dérouler en montrant le dashboard)** : 1. **Le prix de référence** : le m<sup>2</sup> médian national est de **~2 580 €** (*médiane, pas moyenne — j'y reviens*). 2. **Une France à deux vitesses** : **Paris ~10 250 €/m<sup>2</sup>**, la **Creuse ~1 500 €/m<sup>2</sup>** → un rapport de **~7x**. La carte le montre d'un coup d'œil (rouge = cher, vert = abordable). 3. **Le bien et le budget** : l'**appartement** se vend plus cher au m<sup>2</sup> (~3 340 €) que la **maison** (~2 120 €) — densité urbaine. **72 %** des biens sont « Éco » (< 4 000 €/m<sup>2</sup>), **~5 %** « Luxe » (> 8 000). 4. **La tendance** : sur 36 mois, le prix médian **recule de ~2 %** → le marché s'est **refroidi** récemment (hausse des taux d'intérêt). Insight concret à commenter.

**La phrase de fin** : « Le but n'était pas d'afficher des chiffres, mais de les rendre *parlants* : on lit le marché immobilier français en quelques secondes. »

## 2) POURQUOI CETTE DATA / À QUOI ELLE SERT (l'énoncé)

- **DVF** = registre public de toutes les mutations immobilières (source : data.gouv.fr / DGFIP).
- **À quoi ça sert** : estimer un bien, comparer des territoires, suivre la conjoncture.
- **L'enjeu de l'énoncé** : « donner de la valeur à la donnée » et la rendre **compréhensible en 10 s**. → réponse : KPIs en haut, carte, classements, et chaque page = **une question simple**.

## 3) LE FLUX COMPLET (à dessiner au tableau si possible)



**Deux briques à assumer clairement** : - **Le pipeline data-engineering** ( [dashboard-dvf/](#) , Python/Pandas/**SQLite**) : je l'ai mis au point sur un **échantillon réel** — le **1er semestre 2025 (~400 000 ventes)** — pour démontrer toute la chaîne de traitement (collecte → base → nettoyage → fusion → features → agrégation). - **Le dashboard final** ( [dashboard-react/](#) ) : pour une vue **nationale sur 3 ans (2,79 M)**, je m'appuie sur les **agrégats de l'API DVF officielle** (médianes mensuelles, par département, par commune) — bien plus pertinent que de charger ~10 Go bruts dans le navigateur.

Si on te demande « pourquoi pas tout en Pandas sur 10 Go ? » → « parce que l'API fournit déjà des agrégats fiables ; côté navigateur on ne charge qu'un JSON de ~100 Ko. C'est un choix d'archi. »

## 4) LE CODE CŒUR (à savoir réexpliquer)

### a) Nettoyage — `dashboard-dvf/src/nettoyage.py`

On charge la table brute depuis **SQLite**, on fiabilise, on resauvegarde la table propre.

```
df = lire("raw_dvf") # pd.read_sql_query("SELECT * FROM raw_dvf", conn)
df = df.drop_duplicates() # doublons exacts
df = df.dropna(subset=["valeur_fonciere", "surface_reelle"]) # valeurs essentielles
df = df[(df["surface_reelle"] >= 5) & (df["surface_reelle"] <= 10000)] # surfaces aberrantes
df = df[(df["valeur_fonciere"] >= 1000) & (df["valeur_fonciere"] <= 50_000_000)] # prix aberrants
df["date_mutation"] = pd.to_datetime(df["date_mutation"], format="%d/%m/%Y")
df = df.reset_index(drop=True)
```

À dire : « *Garbage in, garbage out* : sans nettoyage le dashboard est faux. Je supprime ce qui empêche de calculer un prix au m<sup>2</sup> fiable, et les aberrations (j'avais une vente à 15 milliards €). »

### b) Feature engineering — `src/feature_engineering.py` (le cœur)

```
df["prix_m2"] = (df["valeur_fonciere"] / df["surface_reelle"]).round(2)
df = df[(df["prix_m2"] >= 500) & (df["prix_m2"] <= 15000)] # plage résidentielle réaliste
df["mois"] = df["date_mutation"].dt.month
def segmenter(p):
    if p < 4000: return "Éco"
    elif p <= 8000: return "Standard"
    else: return "Luxe"
df["segment_prix"] = df["prix_m2"].apply(segmenter)
```

À dire : « Le **prix au m<sup>2</sup>** est l'indicateur central : il rend comparables un studio et une villa. Je crée aussi le mois (pour l'évolution) et un **segment métier** Éco/Standard/Luxe. »

### c) Fusion DVF × INSEE — `src/fusion.py` (jointure)

```
df = pd.merge(df_dvf, df_insee, on="code_departement", how="left")
```

À dire : « Je **croise** les prix (DVF) avec la **population** (INSEE) pour calculer un indicateur de dynamisme : *ventes pour 100 000 habitants*. Je joins sur `code_departement` car les **noms de communes ne sont pas normalisés** dans DVF (« MARSEILLE 1ER »...) → une clé peu fiable donne une fusion ratée (c'est la leçon du cours). **LEFT join** = je garde toutes les ventes même sans correspondance. »

### d) Agrégation — `src/agregation.py` (groupby)

```
g = df.groupby("code_departement").agg(
    prix_median=("prix_m2", "median"),
    prix_moyen=("prix_m2", "mean"),
    nb=("prix_m2", "size").reset_index()
```

À dire : « Le **groupby** passe de l'analyse ligne par ligne à des **stats par groupe** (ici par département). Idem par commune, par type de bien, par mois. C'est ce qui alimente carte et classements. »

#### e) Restitution 3 ans via l'API — `export_react_data.py`

```
nation = get(f"{API}/nation")["data"][0]           # agrégat national
mois   = get(f"{API}/nation/mois")["data"][-36:]    # 36 derniers mois = 3 ans
dist   = get(f"{API}/distribution/nation")          # histogramme des prix -> segments
deps   = get(f"{API}/departement")["data"]          # tous les départements
# médiane = m_am (tous biens), m_a (appart), m_m (maison) ; volume = am/a/m
```

À dire : « Je récupère les agrégats officiels, je découpe la série sur 36 mois, et je calcule les parts Éco/Standard/Luxe à partir de l'histogramme de distribution des prix. Sortie : un seul `data.json`. »

#### f) Front — filtre qui recalcule tout ( `dashboard-react/src/App.tsx` )

```
const view = data[slice]; // slice = "Tous" | "Appartement" | "Maison"
```

À dire : « Tout le dashboard lit `view`. Quand je change le **type de bien**, `view` change et **tous** les KPIs, la carte, les graphes et les tableaux se recalculent. »

#### g) La carte choroplèthe — `src/FranceMap.tsx` (`d3-geo`)

```
const projection = geoMercator().fitSize([600, 560], geo); // adapte la projection au GeoJSON
const path = geoPath(projection);                          // -> "d" de chaque <path> SVG
// couleur = palier de prix médian (vert < 2500 ... rouge > 6000)
```

À dire : « Je projette le GeoJSON des départements en SVG avec **d3-geo**, et je colore chaque département selon son **prix médian** (échelle de couleurs). Clic = détail. »

#### h) Le jeu — `src/DataGame.tsx` (la donnée EST le terrain)

```
const groundAt = (wx) => H - GROUND - ((priceAt(wx) - min) / span) * AMP;
// le sol suit la courbe des prix : haut = cher, bas = abordable ; 1 pièce = 1 mois
```

À dire : « Le **relief du niveau** = la **courbe d'évolution des prix**. En courant, on lit chaque mois (prix affiché sur les pièces), la **couleur** indique hausse/baisse, et l'arrivée donne un **rapport de données**. C'est une façon ludique de *faire vivre* la donnée. »

### 5) DÉMO — ordre conseillé (garde-la pour le moment fort)

1. **Vue d'ensemble** : lis les 4-5 KPIs + commente l'évolution (-2 %).
2. **Carte** (Géographie) : « Paris/Côte d'Azur en rouge » → clique un département → détail.
3. **Bascule Appartement/Maison** (en haut) → tout se recalcule en direct.
4. **Communes** : tape une ville dans la recherche / filtre par département / trie une colonne.
5. **Données & Méthode** : montre le **pipeline** (ton argument technique).
6. (Bonus si le temps / l'ambiance s'y prête) thème **Mario** + **Le Jeu** → effet « waouh » maîtrisé.
7. **Le code** : ouvre `feature_engineering.py` (prix\_m2) et `fusion.py` (merge) et explique.

### 6) QUESTIONS PROBABLES — réponses

**Médiane vs moyenne ?** La médiane = valeur du milieu (50/50), **insensible aux extrêmes** (un château ne la déforme pas). Sur DVF, médiane ≈ 2 580 € mais moyenne plus haute → preuve qu'il y a des valeurs extrêmes. Pour un prix « typique », la médiane est plus honnête.

**Comment tu calcules le prix au m² ?** `valeur_fonciere / surface_reelle` , borné 500–15 000 €/m² pour écarter les aberrations (ventes multi-lots, erreurs de saisie).

**Pourquoi supprimer des lignes ?** ~ce qui est inexploitable : pas de prix/surface, doublons, montants impossibles. Je l'assume et je l'affiche (sur l'échantillon : 404 868 → 373 038).

**C'est quoi un groupby ?** Regrouper les lignes par une clé (département, commune, mois) et calculer une stat par groupe (médiane, nb). `df.groupby("code_departement")["prix_m2"].median()` .

**SQLite, pourquoi ?** Centraliser au lieu de fichiers éparpillés ; base locale légère. Pandas lit avec `pd.read_sql_query` . Tables : `raw_dvf` , `raw_insee` , `dvf_clean` .

**Sur quelle clé tu fusionnes et pourquoi ?** `code_departement` (clé fiable) ; les noms de communes DVF ne sont pas standardisés → fusion ratée. LEFT join pour ne perdre aucune vente.

**Pourquoi React et pas Streamlit/HTML simple ?** Pour des composants pro, des filtres réactifs et une vraie carte interactive (d3-geo). C'est du JS/TS → conforme à l'option « front HTML/CSS/JS » de l'énoncé.

**Pourquoi 3 ans et pas 5 ?** Lisibilité : 3 ans suffisent pour montrer la tendance récente sans noyer l'analyse. La courbe couvre 36 mois.

**Le dashboard charge-t-il les 10 Go ?** Non : le navigateur ne charge qu'un **JSON de ~100 Ko** d'agréats. Choix d'architecture (performance + pertinence).

**As-tu utilisé l'IA ?** Oui pour accélérer le code (c'était autorisé), mais **je comprends et je peux réexpliquer chaque partie** — *(et tu le prouves en ouvrant un fichier)*.

## 7) CHIFFRES À CONNAÎTRE PAR CŒUR

- **2,79 M** transactions · **2023–2025** · **93** départements · ~38 700 communes.
- Médian national **~2 580 €/m²** · appart **~3 340** · maison **~2 120**.
- Paris **#1 (~10 250 €/m²)** · Creuse la plus abordable (**~1 500**) · top commune **Ramatuelle (~16 100)**.
- Segments : **Éco ~72 %** · Standard ~22 % · **Luxe ~5 %** · évolution 3 ans **≈ -2 %**.
- Échantillon pipeline : **404 868 → 373 038** lignes après nettoyage.

## 8) PLAN MINUTÉ (≈10 min) + CHECK-LIST

| Temps    | Contenu                                                      |
|----------|--------------------------------------------------------------|
| 0–1 min  | Accroche + problématique (lisible en 10 s)                   |
| 1–3 min  | La data DVF + le flux/pipeline (schéma)                      |
| 3–7 min  | <b>Démo</b> : KPIs → carte → filtres → communes → méthode    |
| 7–9 min  | Code cœur (prix_m2, fusion, groupby) + insights/storytelling |
| 9–10 min | Conclusion (ce que ça raconte du marché) + ouverture         |

**Avant de passer :** `cd dashboard-react && bun run dev` (onglet ouvert) · éditeur prêt sur `src/` · connaître les 6 chiffres clés · respirer, parler lentement, regarder le jury. **Tu connais ton projet.**